

Aletheia Three.js Animation

An extensive hobby-developer how-to

This guide explains how the Aletheia Three.js Animation was developed and how its parts fit together. The app combines Three.js, WebGL point rendering, Canvas 2D text sampling, a sprite-based sun, pointer interaction, CSS perspective, and ordinary HTML controls.

Reference app: aletheia-threejs-animation.htm

Companion recreation specification: aletheia-threejs-animation-code.md

Current preferred debug defaults:

Universe 1.00

Particles 0.30

Camera 0.50

Sun 4.0

The central idea is simple: use one cloud of particles and continuously steer those same particles toward different target coordinates. The stars are not replaced when the display changes from Spiral to AI to Aletheia. Their destinations change.

1. Architecture: use each browser technology for what it

The page is deliberately hybrid rather than forcing every visual element into WebGL.

Three.js and WebGL render the animated universe. BufferGeometry stores thousands of point coordinates. ShaderMaterial controls the appearance of each glowing point. A PerspectiveCamera gives the cloud real 3D depth.

Canvas 2D is used as a helper. It paints the artificial sun texture and rasterises AI and Aletheia so visible text pixels can be converted into particle destinations.

HTML and CSS render the title, Debug button, sliders, stage buttons, live debug summary, and the perspective crawl. Keeping readable interface text outside WebGL makes it easier to style, edit and debug.

Useful references:

<https://threejs.org/docs/>

<https://threejs.org/docs/pages/WebGLRenderer.html>

<https://threejs.org/docs/pages/BufferGeometry.html>

2. Scene, renderer and camera

The app starts with a WebGLRenderer attached to a full-window canvas. Antialiasing is enabled and the renderer uses an alpha channel so the CSS background can remain visible.

The device pixel ratio is capped at 2. This is an important performance compromise. Rendering at an unrestricted high-DPI pixel ratio can multiply the GPU workload without adding much visible benefit to a field of tiny stars.

A PerspectiveCamera is used with a 48 degree field of view. The near and far planes are wide enough to allow the point cloud and background stars to move in depth.

The animated point cloud and sun are placed inside a THREE.Group called the universe. Dragging rotates the group instead of recomputing every target coordinate.

Core pattern:

```
const renderer = new THREE.WebGLRenderer({ canvas, antialias:true, alpha:true });
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(48, 1, 0.05, 5000);
const universe = new THREE.Group();
scene.add(universe);
```

Reference:

<https://threejs.org/docs/pages/PerspectiveCamera.html>

3. The particle point cloud

The main effect uses about 5,600 points. Three typed arrays hold the per-particle data: position contains x, y and z; aSize contains a point-size value; color contains RGB values.

These arrays are attached to THREE.BufferGeometry. BufferGeometry is a natural fit because the animation is fundamentally a large set of numbers that must be updated and sent to the GPU efficiently.

A custom ShaderMaterial turns each point into a luminous round particle. In the vertex shader, point size is adjusted for perspective. In the fragment shader, distance from gl_PointCoord to the point centre is used to create a bright core and softer glow.

AdditiveBlending is important. When bright particles overlap, their light adds together. Dense regions therefore look naturally brighter than sparse regions.

The colour mix is intentionally limited: white, cyan and pale blue. This keeps the star field coherent while still giving individual stars variation.

Reference:

<https://threejs.org/docs/pages/Points.html>

<https://threejs.org/docs/pages/ShaderMaterial.html>

4. Building the three-arm spiral

The spiral went through several iterations. A mathematically neat spiral was not enough because the desired visual behaviour was specific: strong curl close to the centre, then progressively straighter arms farther out.

The final generator has three principal arms. A proportion of particles are offset into nearby sibling branches. This creates the impression that a stream can split without requiring six completely separate arms.

A parameter t runs from 0 near the centre to 1 near the outside. Radius grows with t . The important part is the angular curve:

```
curve = thetaMax * Math.pow(1 - t, 1.28)
```

Near the centre, $1 - t$ is large, so the arm receives most of its angular turn. Near the outside, $1 - t$ approaches zero, so the turn fades and the arm becomes straighter.

Small Gaussian random offsets are added to angle, radius and depth. That widens each star stream and stops the spiral looking as though it was drawn with a perfect compass.

This is a visual spiral rather than a physical galaxy simulation. The equations are tuned for the desired screen composition.

Related mathematical idea: an Archimedean spiral normally has radius increasing linearly with angle. Here the visual design modifies angular behaviour so the outer flow straightens.

5. Turning AI into a cloud of stars

The AI shape is generated with an off-screen Canvas 2D element. The letters AI are drawn in white using a heavy sans-serif font.

`CanvasRenderingContext2D.getImageData()` exposes the RGBA pixels. The program steps across the canvas and records only pixels whose alpha value is high enough to belong to the visible letters.

Those visible pixel coordinates become a pool of possible particle destinations. For each particle, one text pixel is selected and mapped from canvas pixels into the app's x/y coordinate system. A small random z value gives the letters depth.

This approach has two useful properties. First, it avoids the complexity of font outline geometry. Second, almost any browser-renderable font can become a particle font.

The heavy typeface also gives the I a real top and base rather than making it look like a plain vertical bar.

References:

<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/.fillText>

[https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D.getImageData](https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/getImageData)

6. Turning Aletheia into a classical particle word

Aletheia uses the same raster-to-particle method as AI but a different font treatment.

The displayed word remains Aletheia in Latin letters. The design goal was a Greek or classical visual character, not Greek alphabet substitution. Cinzel is loaded from Google Fonts and used first, with Palatino, Book Antiqua and Times New Roman as fallbacks.

The text is sampled more densely than AI so the longer word retains enough definition. The target width is also larger so Aletheia can span much of the screen.

A small z spread prevents the word from becoming a perfectly flat sheet. When the universe is tilted slightly, the word still has a little depth.

Font reference:

<https://fonts.google.com/specimen/Cinzel>

7. Morphing: let the stars do the walking

The Spiral, AI and Aletheia target arrays all contain the same number of coordinates. The point cloud itself is created once.

On each animation frame, the current position moves a fraction of the remaining distance toward the active target:

```
pos[j] += (target[j] - pos[j]) * attraction
```

The attraction factor is calculated with delta time so the transition remains reasonably consistent across different frame rates.

This easing is the visual heart of the animation. If the attraction is too strong, the stars snap into place and the transformation is lost. If it is too weak, the word never becomes clear before the next stage.

Auto mode keeps each stage visible for a while: roughly 7 seconds for Spiral, 9 seconds for AI, and 10 seconds for Aletheia, before returning to Spiral.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>

8. The artificial sun

The central sun is intentionally inexpensive. It is not a sphere mesh, volumetric light or particle simulation.

A hidden 256 by 256 Canvas 2D image paints a radial gradient: white core, warm cream, pale blue glow, then full transparency.

That canvas becomes a `THREE.CanvasTexture`. The texture is placed on a `SpriteMaterial`, and the material is used by a `THREE.Sprite`.

A sprite always faces the camera. That makes it ideal for a glowing star disc: the universe can rotate while the sun remains visually circular.

Additive blending reinforces the glow. A gentle oscillation changes opacity and size slightly over time.

Current preferred sun size is 4.0. Debug includes both a size slider and an on/off checkbox.

References:

<https://threejs.org/docs/pages/Sprite.html>

<https://threejs.org/docs/pages/SpriteMaterial.html>

9. Pointer repulsion and drag rotation

Two different pointer behaviours are implemented.

Dragging changes target rotations for the entire universe group. Actual rotation eases toward those target angles. This feels smoother than assigning rotation directly on every mouse event.

Ordinary pointer movement repels nearby stars. The difficult part is mapping a 2D screen coordinate into the 3D particle coordinate system.

THREE.Raycaster creates a ray from the camera through the pointer. That ray is intersected with an invisible plane passing through the universe centre and facing the camera. The intersection point is then converted into the universe's local coordinates.

Particles inside a small radius are pushed away from that local point. This is not full particle physics; it is a lightweight interactive force designed to feel tactile.

Reference:

<https://threejs.org/docs/pages/Raycaster.html>

10. Camera fitting and the scale-debugging lesson

One of the most useful lessons from development was that apparently missing geometry can simply be incorrectly scaled or badly framed.

The FIT UNIVERSE function calls `computeBoundingSphere()` on the particle geometry. It multiplies the local radius by the universe scale, then derives a camera distance from the `PerspectiveCamera` field of view.

Conceptually:

```
worldRadius = localRadius * universeScale  
cameraZ = worldRadius / tan(fov / 2)
```

A small margin is added so the cloud is not clipped.

The final preferred development defaults are:

Universe 1.00

Particles 0.30

Camera 0.50

Sun 4.0

Mouse-wheel camera changes were removed because accidental wheel movement made the chosen default appear to reset itself.

11. The Star-Wars-style crawl

The gold crawl is not Three.js geometry. It is normal HTML and CSS placed above the WebGL canvas.

The wrapper is transparent, full width and clipped to about 90 percent of the viewport height. It defines CSS perspective. The text block is rotated backward with `rotateX(67deg)`. A keyframe animation translates the block upward.

Because the overlay itself has no opaque background, the universe remains visible behind the words. A mild top gradient acts only as a fade.

The first crawl heading, Aletheia, is deliberately large and heavy - approximately twice the size of the earlier version. The next line, Let AI do the walking, is the main slogan.

The crawl remains ordinary editable HTML text, which makes wording and line lengths much easier to tune than WebGL text.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/CSS/perspective>

12. Top-left title and hidden Debug mode

The public interface is intentionally quiet. At the top left is a small fixed panel with the title Aletheia Three.js Animation, a one-line interaction hint, and a Debug button.

Debug is off by default. Pressing the button toggles a debug-on class on document.body.

CSS selectors hide the scale panel, stage badge, bottom buttons and bottom-right summary while debug-on is absent. This preserves all the useful tuning tools without making the normal animation look like a development console.

The button text changes to Close Debug while the tools are visible.

This is a broadly useful pattern for hobby projects: keep diagnostics inside the app, but hide them behind a deliberate developer control.

13. Bottom-left debug controls

When Debug is enabled, the bottom-left panel exposes the variables that mattered most during visual tuning.

Universe changes the scale of the Three.js universe group. Particles changes the shader's point-size multiplier. Camera changes the multiplier applied to the fitted camera distance. Sun Size changes the sprite scale. Sun on/off toggles the sprite.

The quick universe buttons are 0.5x, 0.7x and 1x. The earlier 10x experiment was removed because it compressed the useful composition and made the star field appear misleadingly dense.

FIT UNIVERSE recalculates framing. RESET VIEW restores Universe 1.00, Particles 0.30, Camera 0.50 and Sun 4.0.

Below the controls, a diagnostic block reports stage, universe scale, particle size, camera z, sun size, local radius, world radius and the point count.

14. Bottom-middle controls and bottom-right live summary

The bottom-middle buttons are mainly development and demonstration tools.

Spiral, AI and Aletheia force a particular target shape. Auto resumes the timed cycle. Replay returns to Spiral. Crawl toggles the perspective text. Fullscreen calls the browser Fullscreen API.

The bottom-right summary is deliberately compact:

Debug - Universe 1.00 / Particles 0.30 / Camera 0.50 / Sun 4.0

An earlier implementation made this line static. That was a real bug: sliders changed the animation, but the summary did not change.

The fix is a general UI rule: if text represents application state, generate it from the live variables instead of maintaining a second hard-coded copy.

The `updateDebugHint()` function is called whenever Universe, Particles, Camera, Sun Size or Sun on/off changes.

15. Performance, portability and next steps

The application deliberately uses a modest point count and a single `THREE.Points` object. The same point cloud is reused for every morph target.

Performance choices include: capped device pixel ratio, typed arrays, one `requestAnimationFrame` loop, a sprite rather than a volumetric sun, HTML/CSS for text overlays, and a separate mostly-static background star field.

The current file can run directly from disk as an `.htm` file, but it needs internet access for the `Three.js` CDN and Google Font. A fully offline edition could bundle those dependencies locally.

Possible future enhancements include adaptive point count for slower devices, a `prefers-reduced-motion` mode, GPU-side particle interpolation, additional target words or symbols, and a small editor for creating new text or shape targets.

The key lesson is not the specific spiral or font. It is the architecture: one simple point system, several target coordinate sets, and browser technologies used where each is strongest.

16. Reference URLs

Three.js documentation

<https://threejs.org/docs/>

WebGLRenderer

<https://threejs.org/docs/pages/WebGLRenderer.html>

Points

<https://threejs.org/docs/pages/Points.html>

BufferGeometry

<https://threejs.org/docs/pages/BufferGeometry.html>

ShaderMaterial

<https://threejs.org/docs/pages/ShaderMaterial.html>

PerspectiveCamera

<https://threejs.org/docs/pages/PerspectiveCamera.html>

Raycaster

<https://threejs.org/docs/pages/Raycaster.html>

Sprite and SpriteMaterial

<https://threejs.org/docs/pages/Sprite.html>

<https://threejs.org/docs/pages/SpriteMaterial.html>

Canvas fillText and getImageData

<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/fillText>

<https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/getImageData>

requestAnimationFrame

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>

CSS perspective

<https://developer.mozilla.org/en-US/docs/Web/CSS/perspective>

Cinzel font

<https://fonts.google.com/specimen/Cinzel>